Федеральное государственное автономное образовательное учреждение высшего образования **«Национальный исследовательский университет ИТМО»**

Факультет Программной Инженерии и Компьютерной Техники

Лабораторная работа **№4**
«Аппроксимация функции методом наименьших квадратов»

по дисциплине «Вычислительная математика»

Вариант: **8**

Преподаватель:
Зинченко Антон Андреевич

Выполнил:
Некрутенко Максим Владимирович
Группа: P3206

Санкт-Петербург, 2026 г.

Цель работы: найти функцию, являющуюся наилучшим приближением заданной табличной функции по методу наименьших квадратов.

1. Вычислительная реализация задачи

Линейная аппроксимация:

$y = \frac{3x}{x^4 + 8}$	$x \in [-2, 0] \quad h = 0{,}2$

n = 11

i	1	2	3	4	5	6	7	8	9	10	11
x_i	-2	-1.8	-1.6	-1.4	-1.2	-1	-0.8	-0.6	-0.4	-0.2	0
y_i	-0.250	-0.292	-0.330	-0.355	-0.357	-0.333	-0.285	-0.221	-0.150	-0.075	0

φ(x) = a + bx

Вычисляем суммы: sx = -11, sxx = 15.40, sy = -2.648 sxy = 3.2476

$$\begin{cases} 15,40a + (-11,0)b = 3,2476 \\ -11,0a + 11b = -2,648 \end{cases}$$

$11b=11a-2{,}648$

b = a - 0,240727

$b=a-0{,}240727$

$15{,}40a-11{,}0(a-0{,}240727)=3{,}2476$

15,40a - 11,0a + 2,648 = 3,2476

$15{,}40a - 11{,}0a + 2{,}648 = 3{,}2476$

4,40a = 0,5996

$4{,}40a = 0{,}5996$

$a = 0{,}13627$

b = -0,10446

$b = -0{,}10446$

$\varphi(x) = 0{,}13627x - 0{,}10446$

i	1	2	3	4	5	6	7	8	9	10	11
x_i	-2	-1.8	-1.6	-1.4	-1.2	-1	-0.8	-0.6	-0.4	-0.2	0
y_i	-0.250	-0.292	-0.330	-0.355	-0.357	-0.333	-0.285	-0.221	-0.150	-0.075	0
φ(xi)	-0.377	-0.349	-0.322	-0.295	-0.267	-0.240	-0.213	-0.186	-0.158	-0.131	-0104
(φ (xi)-yi)^2	0.016	0.0033	0.000056	0.0035	0.0079	0.0085	0.0051	0.0012	0.000080	0.0032	0.0109

$$\sigma = \sqrt{\frac{\sum(\varphi\,(xi) - yi)^2}{n}} = \mathbf{0.074}$$

<u>Квадратичная аппроксимация</u>:

$y = \dfrac{3x}{x^4 + 8}$	$x \in [-2, 0] \quad h = 0{,}2$

n = 11

i	1	2	3	4	5	6	7	8	9	10	11
x_i	-2	-1.8	-1.6	-1.4	-1.2	-1	-0.8	-0.6	-0.4	-0.2	0
y_i	-0.250	-0.292	-0.330	-0.355	-0.357	-0.333	-0.285	-0.221	-0.150	-0.075	0

$\varphi(x) = a + bx + cx^2$

Вычисляем суммы:
sx = -11, sxx = 15.4, sxxx = -24.2, sxxxx = 40.5328, sy = -2.648, sxy = 3.2476, sxxy = -4.62

$$\begin{cases} 11a_0 - 11,0a_1 + 15,40a_2 = -2,648 \\ -11,0a_0 + 15,40a_1 - 24,20a_2 = 3,2476 \\ 15,40a_0 - 24,20a_1 + 40,5328a_2 = -4,62272 \end{cases}$$

По методу Крамера:
$\Delta = 65.1872$
$\Delta_1 = 1.271,\ \Delta_2 = 35.82,\ \Delta_3 = 13.47$

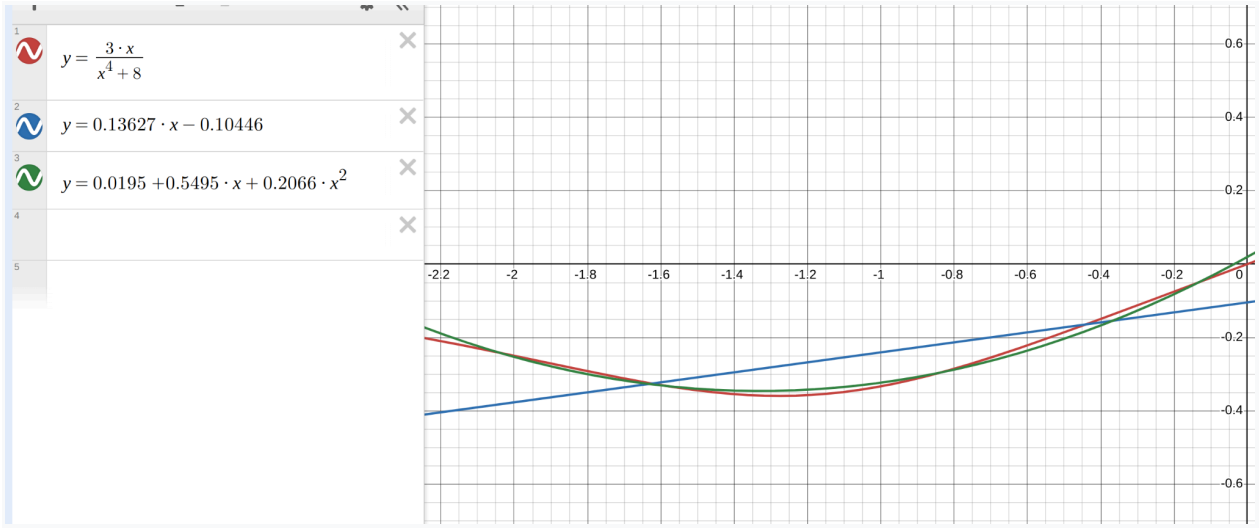
$\{a = \frac{\Delta_1}{\Delta} \approx 0.0195\ \ b = \frac{\Delta_2}{\Delta} \approx 0.5495\ c = \frac{\Delta_3}{\Delta} \approx 0.2066$

$$\varphi_{quad}(x) = 0,0195 + 0,5495x + 0,2066x^2$$

i	1	2	3	4	5	6	7	8	9	10	11
x_i	-2	-1.8	-1.6	-1.4	-1.2	-1	-0.8	-0.6	-0.4	-0.2	0
y_i	-0.250	-0.292	-0.330	-0.355	-0.357	-0.333	-0.285	-0.221	-0.150	-0.075	0
φ(xi)	-0.2531	-0.3002	-0.3308	-0.3449	-0.3424	-0.3234	-0.2879	-0.2358	-0.1672	-0.0821	0.0195
(φ (xi)-yi)^2	0.000009 6	0.0000672	0.000006	0.0001020	0.000213 2	0.0000922	0.0000084	0.0002190	0.0002958	0.0000504	0.0003803

$$\sigma = \sqrt{\frac{\sum(\varphi\ (xi) - yi)^2}{n}} = \mathbf{0.011}$$

0.074 > 0.011, у квадратичной аппроксимации среднеквадратичное отклонение меньше, поэтому это приближение лучше.


1
$y = \frac{3 \cdot x}{x^4 + 8}$
2
$y = 0.13627 \cdot x - 0.10446$
3
$y = 0.0195 + 0.5495 \cdot x + 0.2066 \cdot x^2$
4
5
0.6
0.4
0.2
-0.2
-0.4
-0.6
-2.2
-2
-1.8
-1.6
-1.4
-1.2
-1
-0.8
-0.6
-0.4
-0.2
0

2. Программная реализация задачи

```
def solve_gauss(A, B):
    n = len(B)

    a = [row[:] for row in A]
    b = B[:]

    for k in range(n - 1):
        max_val = abs(a[k][k])
        max_row = k
        for i in range(k + 1, n):
            if abs(a[i][k]) > max_val:
                max_val = abs(a[i][k])
                max_row = i
        if max_val < 1e-12:
            print("Система вырождена или несовместна.")
            return None

        if max_row != k:
            a[k], a[max_row] = a[max_row], a[k]
            b[k], b[max_row] = b[max_row], b[k]

        for i in range(k + 1, n):
            factor = a[i][k] / a[k][k]
            for j in range(k, n):
                a[i][j] -= factor * a[k][j]
            b[i] -= factor * b[k]

        if abs(a[n - 1][n - 1]) < 1e-12:
            print("Система вырождена. Последний диагональный
элемент = 0.")
            return None

    x = [0.0] * n
    for i in range(n - 1, -1, -1):
        sum_ax = 0.0
        for j in range(i + 1, n):
            sum_ax += a[i][j] * x[j]

        x[i] = (b[i] - sum_ax) / a[i][i]

    return x
```

```
def linear(x, y):
    n = len(x)
    sx, sy = sum(x), sum(y)
    sxx = sum(xi ** 2 for xi in x)
    sxy = sum(xi * yi for xi, yi in zip(x, y))
    det = n * sxx - sx ** 2

    if abs(det) < 1e-12:
```

```
        return None

    a = (n * sxy - sx * sy) / det
    b = (sxx * sy - sx * sxy) / det
    return (a, b)


def polyn2degree(x, y):
    n = len(x)

    sx = sum(x)
    sxx = sum(xi ** 2 for xi in x)
    sxxx = sum(xi ** 3 for xi in x)
    sxxxx = sum(xi ** 4 for xi in x)

    sy = sum(y)
    sxy = sum(xi * yi for xi, yi in zip(x, y))
    sxxy = sum((xi ** 2) * yi for xi, yi in zip(x, y))

    A = [
        [n, sx, sxx],
        [sx, sxx, sxxx],
        [sxx, sxxx, sxxxx]
    ]

    B = [sy, sxy, sxxy]

    return solve_gauss(A, B)


def polyn3degree(x, y):
    n = len(x)

    sx = sum(x)
    sxx = sum(xi ** 2 for xi in x)
    sxxx = sum(xi ** 3 for xi in x)
    sxxxx = sum(xi ** 4 for xi in x)
    sxxxxx = sum(xi ** 5 for xi in x)
    sxxxxxx = sum(xi ** 6 for xi in x)

    sy = sum(y)
    sxy = sum(xi * yi for xi, yi in zip(x, y))
    sxxy = sum((xi ** 2) * yi for xi, yi in zip(x, y))
    sxxxy = sum((xi ** 3) * yi for xi, yi in zip(x, y))

    A = [
        [n, sx, sxx, sxxx],
        [sx, sxx, sxxx, sxxxx],
        [sxx, sxxx, sxxxx, sxxxxx],
        [sxxx, sxxxx, sxxxxx, sxxxxxx]
    ]

    B = [sy, sxy, sxxy, sxxxy]
```

```
    return solve_gauss(A, B)


def logarifm(x, y):
    if any(xi <= 0 for xi in x):
        return None

    ln_x = [math.log(xi) for xi in x]
    return linear(ln_x, y)


def exponential(x, y):
    if any(yi <= 0 for yi in y):
        return None
    ln_y = [math.log(yi) for yi in y]
    res = linear(x, ln_y)
    if res is None:
        return None
    A, B = res
    return (math.exp(A), B)


def power(x, y):
    if any(xi <= 0 for xi in x) or any(yi <= 0 for yi in y):
        return None
    ln_x = [math.log(xi) for xi in x]
    ln_y = [math.log(yi) for yi in y]
    res = linear(ln_x, ln_y)
    if res is None:
        return None
    A, B = res
    return (math.exp(A), B)


def calc_delta(y_true, y_pred):
    n = len(y_true)
    return math.sqrt(sum((p - t) ** 2 for p, t in zip(y_pred,
y_true)) / n)


def calc_r2(y_true, y_pred):
    mean_y = sum(y_true) / len(y_true)
    ss_res = sum((t - p) ** 2 for t, p in zip(y_true, y_pred))
    ss_tot = sum((t - mean_y) ** 2 for t in y_true)
    if ss_tot < 1e-12:
        return 0.0
    return 1 - ss_res / ss_tot


def calc_pearson(x, y):
    n = len(x)
    mx, my = sum(x) / n, sum(y) / n
```

```python
    num = sum((xi - mx) * (yi - my) for xi, yi in zip(x, y))
    dx = math.sqrt(sum((xi - mx) ** 2 for xi in x))
    dy = math.sqrt(sum((yi - my) ** 2 for yi in y))
    if dx * dy < 1e-12:
        return 0.0
    return num / (dx * dy)


def quality_msg(r2):
    if r2 >= 0.95:
        return "Высокая точность"
    elif r2 >= 0.75:
        return "Удовлетворительная"
    elif r2 >= 0.50:
        return "Слабая"
    else:
        return "Точность недостаточна"


def predict_linear(x, coeffs):
    a, b = coeffs
    return a * x + b


def predict_poly(x, coeffs):
    return sum(c * (x ** i) for i, c in enumerate(coeffs))


def predict_exp(x, coeffs):
    a, b = coeffs
    return a * math.exp(b * x)


def predict_log(x, coeffs):
    a, b = coeffs
    return a * math.log(x) + b


def predict_pow(x, coeffs):
    a, b = coeffs
    return a * (x ** b)
```

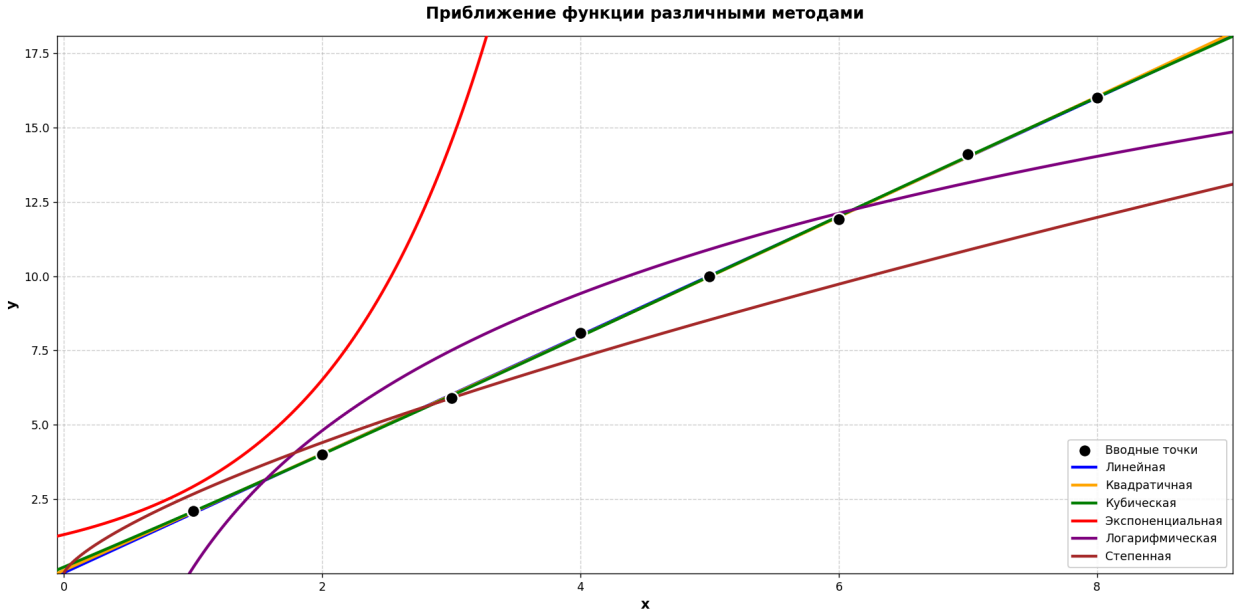
Выберите способ ввода данных:
0. Выход
1. Ввести с клавиатуры
2. Загрузить из файла

Ваш выбор: 1

Введите точки в формате: x y (по одной на строке)
Для завершения введите end:
1 2.1
2 4.0
3 5.9
4 8.1
5 10.0
6 11.9
7 14.1
8 16.0
end
Сохранить результаты в файл? (y/n): n

Модель	δ	R^2	r (лин)	Качество
Линейная	0.0776	0.9997	0.9999	Высокая точность
Квадратичная	0.0736	0.9997	-	Высокая точность
Кубическая	0.0710	0.9998	-	Высокая точность
Экспоненциальная	307.0757	-4504.0419	-	Точность недостаточна
Логарифмическая	1.3348	0.9149	-	Удовлетворительная
Степенная	2.0814	0.7930	-	Удовлетворительная

НАИЛУЧШАЯ: Кубическая
Уравнение: $y = 0.2214 + 1.8312x + 0.0360x^2 + -0.0023x^3$
$\delta = 0.07100$, $R^2 = 0.99976$

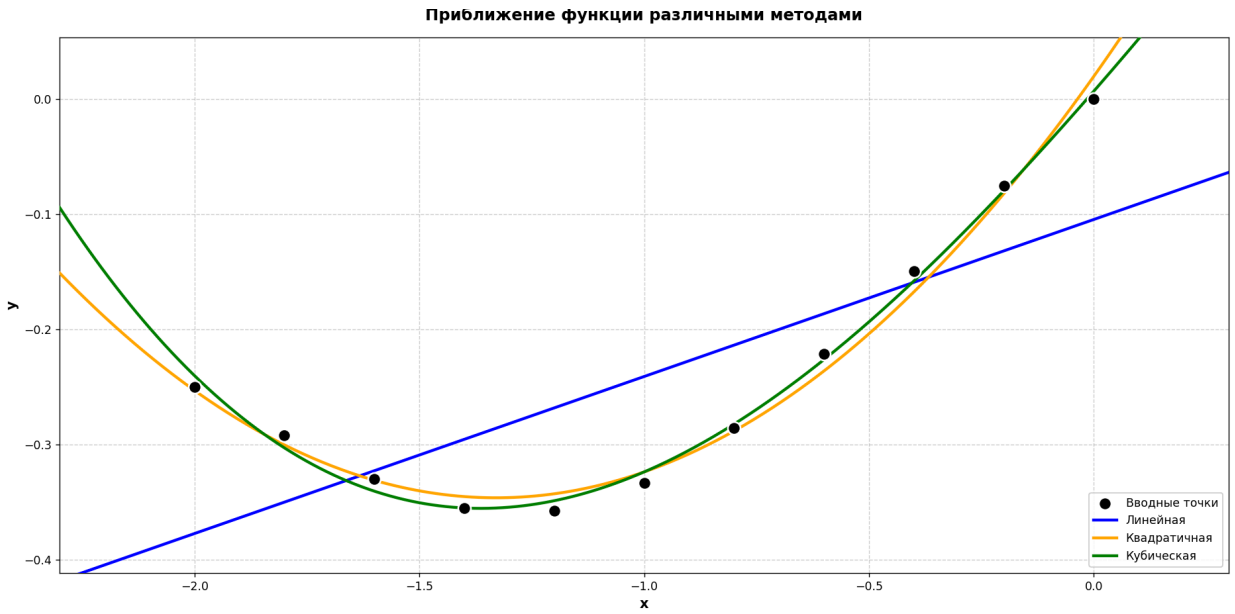


Выберите способ ввода данных:
0. Выход
1. Ввести с клавиатуры
2. Загрузить из файла

Ваш выбор: 2
Введите имя файлаlr4
Сохранить результаты в файл? (y/n): n

Модель	δ	R^2	r (лин)	Качество
Линейная	0.0740	0.5754	0.7586	Слабая
Квадратичная	0.0115	0.9897	-	Высокая точность
Кубическая	0.0078	0.9953	-	Высокая точность

НАИЛУЧШАЯ: Кубическая
Уравнение: $y = 0.0068 + 0.4478x + 0.0728x^2 + -0.0447x^3$
$\delta = 0.00779$, $R^2 = 0.99530$



Выберите способ ввода данных:
0. Выход
1. Ввести с клавиатуры
2. Загрузить из файла

Ваш выбор: 0

Вывод

В процессе данной лабораторной работы была выполнена аппроксимация функций с использованием линейного, квадратичного, кубического, экспоненциального и логарифмического приближений. Также на основе этих методов был реализована программа на Python, которая реализует метод наименьших квадратов и строит графики исходной функции и аппроксимаций. Исследование позволило определить наилучшее приближение, вычислить среднеквадратические отклонения и коэффициент корреляции Пирсона для линейной зависимости.